ChatGPT

Do deep research on collecting representative, realistic OS-level and Oracle log and test-data snippets for a diagnostic framework

Executive summary

A useful Oracle host-diagnostics corpus should not be built around isolated ORA codes. It should be built around **correlated artefacts**: OS/kernel evidence, virtualisation or container/runtime evidence, cloud metrics where available, and the Oracle-facing alert/trace or listener symptom that a DBA actually sees. Oracle's ADR design, Oracle Net logging, Linux PSI/cgroup interfaces, VMware esxtop counters, Kubernetes log locations, Docker event streams, and AWS/GCP/Azure monitoring all support that correlation model. The practical unit of test data is therefore **a small, sanitised snippet bundle + metadata + causal-chain label**, not a single error string. [1]

For framework quality, the snippets should be **small but realistic**: real-looking timestamps, process IDs, device names, mount points, ADR paths, listener log formatting, and cloud metric names where those names are officially documented. Where a provider or platform does not expose a fixed host file path or the reviewed sources do not give an exact metric name, the correct treatment is **"unspecified"**, with a substitute such as guest logs, dashboard export text, or API output. This is especially important for cloud-managed observability streams and for Oracle Net logging when ADR is enabled, because the active log representation may be XML `log.xml` in ADR rather than a legacy flat text file. [2]

The safest and most reusable corpus is therefore **mostly synthetic, but format-faithful**: derive message text, file naming, and counters from primary docs and vendor KBs; then fabricate timestamps, hostnames, PIDs, IPs, and file paths in a repeatable sanitised way. Oracle's Trace File Analyzer explicitly supports consistent masking of hostnames and IP addresses, which is exactly the behaviour a framework dataset should preserve. [3]

```
flowchart LR
    A[Host or cgroup condition]
    B[OS or platform evidence]
    C[Oracle alert or trace]
    D[Client or DBA-facing symptom]
    E[Framework label]

    A --> B
    B --> C
    C --> D
    D --> E

    A::a -->|example| B1["memory.max breach<br/>memory.events: oom_kill=1"]
    B1 --> C1["trace: ORA-04030<br/>alert: process died"]
    C1 --> D1["ORA-03113 / TNS disconnect"]
    D1 --> E1["memory.cgroup_oom_kill"]
```

```
    classDef a fill:#f6f6f6,stroke:#999,color:#222;
```

The flow is representative of how Linux cgroup memory events, Oracle process-memory errors, and Oracle Net disconnect symptoms line up in real incidents. [4]

Log catalog and source priorities

The minimum catalogue below is the recommended **collection contract** for a framework. Paths are listed only where the reviewed primary documentation or vendor KBs are explicit; otherwise the entry is marked **unspecified** and a collector alternative is given.

Category	Required artefact types	Exact filename / location	Notes
Oracle core diagnostics	ADR base and home, database alert, trace, incident, core	`DIAGNOSTIC_DEST/diag/rdbms/<db_name>/<instance_name>`; trace under `<adr-home>/trace`; XML alert under `<adr-home>/alert/log.xml`; incident under `<adr-home>/incident/*`; core under `<adr-home>/cdump`	`V$DIAG_INFO` is the authoritative in-database locator for active ADR paths and the current process's default trace file. Text alert presentation exists, but exact on-disk text alert placement is **unspecified in the reviewed cross-platform ADR sources**; use ADRCI / `V$DIAG_INFO` to resolve it at runtime. [5]
Oracle Net diagnostics	Listener log, listener trace, server/client sqlnet trace, `listener.ora`, `sqlnet.ora`	Listener ADR home path is `diag/tnslsnr/<host>/<listener>`; with ADR enabled, default log file name is `log.xml` in the component's alert directory; default trace file names include `listener.trc`, `sqlnet.trc`, and `svr_pid.trc`; `sqlnet.ora` defaults to `$ORACLE_HOME/network/admin`	Legacy `listener.log` is relevant mainly when ADR is disabled; its exact path is configuration-driven and therefore **unspecified without** `listener.ora`. [6]

Category	Required artefact types	Exact filename / location	Notes
Linux host logging	Kernel ring, journald, classic syslog mirror, PSI, meminfo, cgroup files	`dmesg`; journald persistent `/var/log/journal`, volatile `/run/log/journal`; classic flat-file syslog location is distro-specific; on many RHEL-family systems relevant messages also surface in `/var/log/messages`; PSI files under `/proc/pressure/{cpu,memory,io}`; cgroup v2 such as `memory.events` and `memory.pressure`	Prefer journald + exported text, because flat-file syslog paths are not universal. [7]
CPU issue support data	`vmstat`, `mpstat`, `pidstat`, PSI, hypervisor counters	Tool output files are user-defined; no fixed OS filename	CPU saturation often has **telemetry, not kernel error lines**. For VMs, capture VMware `%RDY/%CSTP/%MLMTD` or KVM steal `%st`. [8]
Memory issue support data	`/proc/meminfo`, `vmstat`, swap, OOM logs, cgroup OOM counters, VM balloon/swap counters	`/proc/meminfo`; journald / dmesg; cgroup v2 `memory.events`; VMware esxtop memory panels / batch output filename user-defined	`memory.events` values `high`, `max`, `oom`, `oom_kill` are canonical for container and cgroup pressure. [9]
Disk and I/O issue support data	Kernel I/O errors, `iostat`, PSI I/O, VM storage counters, cloud disk metrics, Oracle SQL trace waits	`dmesg` / journald; user-defined `iostat` capture; VMware `esxtop` batch export user-defined; AWS CloudWatch / GCP Cloud Monitoring / Azure Monitor have **no fixed local file path** unless exported	Officially documented examples include `blk_update_request: I/O error`, ESXi `DAVG/KAVG/GAVG`, AWS EBS queue and latency metrics, GCP queue depth/latency dashboard labels, and Azure disk queue-depth/latency metrics. [10]
Kernel and system-limit support data	`ulimit`, `sysctl`, `ipcs`, sem/shm data, HugePages state, ADR error stacks	Command output paths are user-defined; HugePages status via `/proc/meminfo`; shell limits via process/session environment	Oracle's OS-call failure surface for these incidents is commonly `ORA-27300/27301/27302` or, for memory-related startup failures, `ORA-27102`. [11]

Category	Required artefact types	Exact filename / location	Notes
Network support data	Listener log, server/client sqlnet traces, conntrack messages, `ss`, `sar -n`, NIC counters, VM/network counters, cloud metrics	Listener ADR paths as above; `nf_conntrack` messages in kernel log / syslog; Kubernetes node logs in journald or `/var/log`; pod/container logs under `/var/log/pods`; Docker event stream via `docker events`; cloud metrics via service APIs / dashboards, local path **unspecified**	Node and pod/container logging locations are different in Kubernetes; capture both. [12]
VMware-specific artefacts	`esxtop` CPU/memory/network/storage counters, `vmkernel.log`, optionally VM `vmware.log`	ESXi host logs under `/var/log`, including `/var/log/vmkernel.log`; VM log file `vmware.log` in the VM directory; `esxtop` batch export filename is user-chosen	For network, `%DRPTX`; for CPU, `%RDY/%CSTP`; for storage, `DAVG/KAVG/GAVG`; for memory, `SWCUR` and balloon-related signals. [13]
KVM-specific artefacts	Guest steal time, guest virtio characteristics, guest ballooning, optional `kvm_stat`	Steal time appears in `/proc/stat`, `top`, `vmstat`; balloon visibility is hypervisor / guest specific; `kvm_stat` output path is user-defined	High `%st` is the key guest symptom; virtio is the recommended high-performance storage/network path. [14]
Container/Kubernetes artefacts	Pod logs, previous container logs, events, node journald, cgroup stats, Docker writable-layer context	`/var/log/pods`; node component logs in journald or `/var/log`; `kubectl logs -p`; `kubectl events`; Docker/Kubernetes event streams no fixed local filename unless exported	Write-heavy database data should not sit in a Docker writable layer; use volumes or PVs. [15]

Category	Required artefact types	Exact filename / location	Notes
Cloud metrics	Disk latency/queue depth, network allowance/ bandwidth ceilings, instance limits	CloudWatch / Cloud Monitoring / Azure Monitor APIs or dashboard exports; local filename **unspecified** unless you export to CSV/ JSON	Exact metric names are best treated as provider-specific. AWS names are well documented in the reviewed sources; some GCP/Azure dashboard labels are documented, but exact API metric names for every example are partly **unspecified in the reviewed sources**. [16]

Primary-source priority for building the corpus should be: Oracle ADR / Error Help / Oracle Net docs first; Linux kernel docs second; VMware Broadcom KB and VMware docs third; AWS/GCP/Azure monitoring docs fourth; Kubernetes and Docker docs fifth; Red Hat KVM and Linux KBs sixth. Third-party blogs should be fallback only, and any snippet built from them should be marked lower priority. [17]

Representative snippet corpus

All snippets below are either **synthetic** or **sanitised synthetic composites**. None contains real production identifiers. Each snippet is intentionally short, unit-test friendly, and traceable to a likely causal chain.

CPU snippets

CPU saturation is usually evidenced by telemetry and scheduling delay rather than a crisp kernel error. Linux PSI is authoritative for scheduler pressure, VMware exposes `%RDY/%CSTP/%MLMTD`, KVM exposes steal `%st`, and cgroup CPU throttling is visible in `cpu.stat`. Oracle commonly surfaces timeouts such as `ORA-3136`, `TNS-12535`, or `ORA-12547` rather than a CPU-specific ORA. [18]

Issue: Host CPU saturation causing inbound connect timeout
Example snippet ID: `CPU-01`
Environment: physical bare-metal
Type: Synthetic
Why representative: Built from Linux PSI + `vmstat` style + Oracle Net timeout text; realistic because CPU contention often shows as pressure and timeout, not a direct kernel error. [19]

```
2026-04-24T09:14:20+05:30 host-a cpu.pressure: some avg10=72.14 avg60=48.92
avg300=19.50 total=1842332
2026-04-24T09:14:20+05:30 vmstat[2123]: procs -----------memory---------- ---
swap-- -----io---- -system-- ------cpu-----
2026-04-24T09:14:20+05:30 vmstat[2123]:  r  b   swpd   free   buff  cache
si   so    bi    bo   in   cs us sy id wa st
2026-04-24T09:14:20+05:30 vmstat[2123]: 37  0      0  23120  90432 1442208
0    0     1    12 41211 80123 95  4  1  0  0
2026-04-24T09:14:21+05:30 pidstat[2124]: 09:14:21 UID   PID   %usr %system
%CPU  CPU  Command
```

```
2026-04-24T09:14:21+05:30 pidstat[2124]: 09:14:21 oracle 24173 73.22   2.91
76.13    7 ora_lgwr_FIN1
2026-04-24T09:14:28.338+05:30 WARNING: inbound connection timed out
(ORA-3136)
24-APR-2026 09:14:29 * (CONNECT_DATA=(SERVICE_NAME=<SERVICE_01>)) * establish
* <SERVICE_01> * 12535
```

Issue: VMware CPU ready / co-stop delaying Oracle processes
Example snippet ID: `CPU-02`
Environment: VMware
Type: Synthetic
Why representative: Uses exact documented esxtop counter names `%RDY`, `%CSTP`, `%MLMTD`; Oracle side shows connect timeout / lost-contact behaviour typical of scheduler delay or late process startup. [20]

```
ESXTOP-CPU 2026-04-24 10:02:00
GID  NAME            %USED  %RUN  %SYS  %RDY  %CSTP  %MLMTD
4711 oracle-db-vm    318.4  246.6  6.1   18.7   7.9     0.0
4712 ora_lgwr_FIN1    98.7   69.5  2.0   19.2   5.1     0.0
4713 ora_pmon_FIN1    99.2   71.3  1.4   16.9   4.8     0.0
2026-04-24T10:02:11.442+05:30 WARNING: inbound connection timed out
(ORA-3136)
24-APR-2026 10:02:12 * (CONNECT_DATA=(SERVICE_NAME=<SERVICE_02>)) * establish
* <SERVICE_02> * 12547
TNS-12547: TNS:lost contact
```

Issue: KVM host overcommit showing high steal time
Example snippet ID: `CPU-03`
Environment: KVM
Type: Synthetic
Why representative: KVM steal time is officially exposed as `%st` in `top` and `vmstat`. The Oracle-facing symptom is deliberately timeout/disconnect oriented, which is more realistic than inventing a CPU-only ORA. [21]

```
2026-04-24T10:33:10+05:30 top[919]: %Cpu(s): 60.8 us, 3.9 sy, 4.0 id, 0.2 wa,
31.1 st
2026-04-24T10:33:10+05:30 vmstat[920]: procs -----------memory---------- ---
swap-- -----io---- -system-- ------cpu-----
2026-04-24T10:33:10+05:30 vmstat[920]:  r  b swpd free buff cache si so bi bo
in cs us sy id wa st
2026-04-24T10:33:10+05:30 vmstat[920]: 21  0    0 18644 81232 923244 0 0 0 3
9231 22102 61 4 4 0 31
2026-04-24T10:33:11+05:30 pidstat[921]: 10:33:11 oracle 25110 67.11 2.45
69.56 3 ora_smon_FIN1
2026-04-24T10:33:19.901+05:30 WARNING: inbound connection timed out
(ORA-3136)
24-APR-2026 10:33:20 * (CONNECT_DATA=(SERVICE_NAME=<SERVICE_03>)) * establish
```

```
* <SERVICE_03> * 12547
ORA-03135: connection lost contact
```

Issue: Container CPU quota throttling causing slow Oracle listener handoff
Example snippet ID: `CPU-04`
Environment: Docker/K8s
Type: Synthetic
Why representative: `cpu.stat` throttling counters are the canonical cgroup signal; the Oracle-facing side remains timeout oriented, which is realistic for quota throttling. [22]

```
# /sys/fs/cgroup/cpu.stat
usage_usec 812944237
user_usec 781220411
system_usec 31723826
nr_periods 14402
nr_throttled 13877
throttled_usec 228492118
2026-04-24T10:49:31.220+05:30 WARNING: inbound connection timed out
(ORA-3136)
24-APR-2026 10:49:32 * (CONNECT_DATA=(SERVICE_NAME=<SERVICE_04>)) * establish
* <SERVICE_04> * 12535
```

Memory snippets

Memory incidents are the best category for strong host-to-Oracle correlation. Linux kernel OOM messages, cgroup `memory.events`, Kubernetes `OOMKilled`, and VMware balloon/swap signals line up cleanly with Oracle `ORA-04030`, `ORA-27102`, and downstream disconnects such as `ORA-03113`. [23]

Issue: Host OOM killer terminated an Oracle server process
Example snippet ID: `MEM-01`
Environment: physical bare-metal
Type: Synthetic
Why representative: Combines canonical Linux OOM-killer wording with Oracle's documented process-memory error and the common client-side EOF symptom. [24]

```
Apr 24 10:12:01 host-a kernel: Out of memory: Kill process 2345 (oracle)
score 987 or sacrifice child
Apr 24 10:12:01 host-a kernel: Killed process 2345 (oracle) total-vm:
2048000kB anon-rss:1536000kB file-rss:0kB shmem-rss:0kB
2026-04-24T10:12:02.114+05:30 Errors in file /u01/app/oracle/diag/rdbms/<DB>/
<INST>/trace/<INST>_ora_2345.trc:
2026-04-24T10:12:02.114+05:30 ORA-04030: out of process memory when trying to
allocate 8192 bytes (pga heap,kxs-heap-c)
2026-04-24T10:12:02.115+05:30 Dumping diagnostic data in
directory=[cdmp_20260424101202], requested by (instance=1, osid=2345)
ORA-03113: end-of-file on communication channel
```

```
Process ID: 2345
Session ID: 123 Serial number: 456
```

Issue: VMware ballooning / guest swap preceding Oracle out-of-memory error
Example snippet ID: `MEM-02`
Environment: VMware
Type: Synthetic
Why representative: Uses documented VMware memory pressure indicators such as ballooning and `SWCUR`, then maps them to Oracle process-private memory exhaustion. [25]

```
ESXTOP-MEM 2026-04-24 11:08:00
GID  NAME            MEMSZ(MB)  GRANT(MB)  MCTL(MB)  SWCUR(MB)  SWTGT(MB)
4711 oracle-db-vm       65536      65536      4096       2048       4096
2026-04-24T11:08:01+05:30 vmstat[3102]:  procs -----------memory----------
---swap-- ------cpu-----
2026-04-24T11:08:01+05:30 vmstat[3102]:  2  0 2097088 68420 90232 921440 0
512 us sy id wa st
2026-04-24T11:08:09.444+05:30 Errors in file /u01/app/oracle/diag/rdbms/<DB>/
<INST>/trace/<INST>_ora_24173.trc:
2026-04-24T11:08:09.444+05:30 ORA-27102: out of memory
2026-04-24T11:08:09.445+05:30 ORA-04030: out of process memory when trying to
allocate 16384 bytes (koh-kghu sessi,kollalo2)
ORA-03113: end-of-file on communication channel
```

Issue: Container memory limit breach and pod OOM kill
Example snippet ID: `MEM-03`
Environment: Docker/K8s
Type: Synthetic
Why representative: Uses Kubernetes termination semantics and documented cgroup v2 `memory.events` fields. This is one of the highest-value snippets for a framework because it is deterministic and compact. [26]

```
Namespace: db
Pod: oracle-db-0
Last State: Terminated
Reason: OOMKilled
Exit Code: 137
# /sys/fs/cgroup/memory.events => high 83 max 14 oom 3 oom_kill 1
2026-04-24T11:42:53.009+05:30 ORA-04030: out of process memory when trying to
allocate 4096 bytes (pga heap,kgsc heap)
ORA-03113: end-of-file on communication channel
```

Issue: Self-managed Azure VM guest OOM; provider host metric name not used
Example snippet ID: `MEM-04`
Environment: Azure
Type: Synthetic
Why representative: On self-managed cloud VMs, the guest Linux OOM signature is usually the most reliable artefact. In the reviewed Azure sources, disk metrics are explicit, but memory-specific host

metric naming for this exact use case is not the key diagnostic source here, so it remains intentionally omitted. [27]

```
Apr 24 06:44:10 ora-az-vm kernel: Out of memory: Kill process 28741 (oracle)
score 913 or sacrifice child
Apr 24 06:44:10 ora-az-vm kernel: Killed process 28741 (oracle) total-vm:
1876000kB anon-rss:1482200kB
2026-04-24T06:44:10.887+05:30 Errors in file /u01/app/oracle/diag/rdbms/<DB>/
<INST>/trace/<INST>_ora_28741.trc:
2026-04-24T06:44:10.887+05:30 ORA-27102: out of memory
2026-04-24T06:44:10.888+05:30 ORA-04030: out of process memory when trying to
allocate 12288 bytes (kgllk,kglnahsh)
ORA-03113: end-of-file on communication channel
Process ID: 28741
Session ID: 431 Serial number: 9912
```

Disk and I/O snippets

I/O incidents split into **latency** and **hard failure**. Linux kernel block/NVMe errors and ESXi `DAVG/KAVG/GAVG` diagnose the host side; Oracle then shows either explicit OS-call failures such as `ORA-27072` / `ORA-01114` or trace wait inflation such as `db file sequential read` and `log file parallel write`. AWS, GCP, and Azure all document disk queue depth and/or latency metrics, but their collection location is API/dashboard export rather than a fixed file on disk. [28]

Issue: NVMe reset and hard I/O error on a physical host
Example snippet ID: `IO-01`
Environment: physical bare-metal
Type: Synthetic
Why representative: Uses documented NVMe reset and `blk_update_request` patterns from Red Hat KBs plus Oracle's documented file-I/O error text. [29]

```
Apr 24 12:01:21 host-a kernel: nvme nvme1: I/O 423 QID 29 timeout, reset
controller
Apr 24 12:01:21 host-a kernel: nvme nvme1: controller is down; will reset:
CSTS=0x3, PCI_STATUS=0x10
Apr 24 12:01:22 host-a kernel: nvme nvme1: Device not ready; aborting reset,
CSTS=0x3
Apr 24 12:01:22 host-a kernel: blk_update_request: I/O error, dev nvme1c1n1,
sector 93500928 op 0x0:(READ)
2026-04-24T12:01:22.703+05:30 Errors in file /u01/app/oracle/diag/rdbms/<DB>/
<INST>/trace/<INST>_ora_24173.trc:
2026-04-24T12:01:22.703+05:30 ORA-27072: File I/O error
2026-04-24T12:01:22.703+05:30 ORA-01114: IO error writing block to file 5
(block # 12345)
Linux-x86_64 Error: 5: Input/output error
```

Issue: ESXi storage latency without immediate hard failure
Example snippet ID: `IO-02`

Environment: VMware
Type: Synthetic
Why representative: Tracks official esxtop storage counters and shows how the Oracle-side evidence is often SQL trace waits, not ORA errors, during a latency-only incident. [30]

```
ESXTOP-STORAGE 2026-04-24 12:16:00
DEVICE                                  DAVG/cmd  KAVG/cmd  GAVG/cmd  QUED
%USD
naa.6000c29f2c9b9b1f5c88f93a8f8a0211       41.3      12.4      53.7    32
100.0
naa.6000c29f2c9b9b1f5c88f93a8f8a0212       38.9      11.1      50.2    29
99.8
WAIT #140311418157416: nam='db file sequential read' ela=128347 file#=7
block#=392112 blocks=1 obj#=94876 tim=1745477160001000
WAIT #140311418157416: nam='db file parallel write' ela=94372 files=3
blocks=24 requests=3 tim=1745477160102000
WAIT #140311418157416: nam='log file parallel write' ela=80511 files=2
blocks=8 requests=2 tim=1745477160183000
WAIT #140311418157416: nam='log file sync' ela=82604 p1=105 p2=0 p3=0 obj#=-1
tim=1745477160185000
```

Issue: Container node ephemeral-storage exhaustion / overlay-backed writes failing
Example snippet ID: `IO-03`
Environment: Docker/K8s
Type: Synthetic
Why representative: Kubernetes documents node/pod log storage and ephemeral storage management; Docker explicitly warns that write-heavy DB storage in writable layers is slower and operationally risky. The Oracle side is usually archive/write failure rather than media error. [31]

```
24s  Warning  Evicted   pod/oracle-db-0  The node was low on resource:
ephemeral-storage.
24s  Normal   Killing   pod/oracle-db-0  Stopping container oracle
Filesystem      Size  Used Avail Use% Mounted on
overlay          40G   40G     0 100% /u01
2026-04-24T12:44:09.550+05:30 Errors in file /u01/app/oracle/diag/rdbms/<DB>/
<INST>/trace/<INST>_ora_33772.trc:
2026-04-24T12:44:09.550+05:30 ORA-19502: write error on file "/u01/app/
oracle/fast_recovery_area/arch_11749.arc", blockno 32769 (blocksize=512)
2026-04-24T12:44:09.551+05:30 ORA-00257: Archiver error. Connect AS SYSDBA
only until resolved.
write failed: No space left on device
```

Issue: AWS EBS queue depth and latency rise driving Oracle wait inflation
Example snippet ID: `IO-04`
Environment: AWS
Type: Synthetic
Why representative: Uses officially documented AWS EBS metric names and Oracle SQL trace wait lines. This is the preferred cloud-storage pattern when host kernel logs are quiet but the block device is simply slow. [32]

```
2026-04-24T07:10:00Z,AWS/EBS,VolumeQueueLength,vol-<VOL01>,Average,42
2026-04-24T07:10:00Z,AWS/EBS,VolumeAvgReadLatency,vol-<VOL01>,Average,0.114
2026-04-24T07:10:00Z,AWS/EBS,VolumeAvgWriteLatency,vol-<VOL01>,Average,0.128
2026-04-24T07:10:00Z,AWS/EBS,VolumeWriteOps,vol-<VOL01>,Sum,18743
WAIT #139882761813744: nam='db file sequential read' ela=112944 file#=8
block#=441002 blocks=1 obj#=90011 tim=1745478300123000
WAIT #139882761813744: nam='db file scattered read' ela=146382 file#=11
block#=118532 blocks=32 obj#=90107 tim=1745478300288000
WAIT #139882761813744: nam='log file parallel write' ela=91872 files=2
blocks=8 requests=2 tim=1745478300392000
WAIT #139882761813744: nam='log file sync' ela=93140 p1=105 p2=0 p3=0 obj#=-1
tim=1745478300395000
```

Kernel and system-limit snippets

This category covers shell limits, sysctls, sem/shm exhaustion, HugePages / `memlock`, and blocked tasks. Oracle's documented OS-call failure surface is `ORA-27300`, with `ORA-27301` carrying the OS message and `ORA-27302` the call-site string when available. For ADR-era systems, validate actual trace location with `V$DIAG_INFO` instead of guessing. [33]

Issue: `nofile` exhaustion causing Oracle open failure
Example snippet ID: `SYS-01`
Environment: physical bare-metal
Type: Synthetic
Why representative: This is one of the cleanest system-limit signatures because the ORA stack carries the OS errno text directly. [34]

```
$ ulimit -n
1024
$ lsof -p 24173 | wc -l
1024
2026-04-24T13:18:42.202+05:30 Errors in file /u01/app/oracle/diag/rdbms/<DB>/
<INST>/trace/<INST>_ora_24173.trc:
2026-04-24T13:18:42.202+05:30 ORA-27300: OS system dependent operation:open
failed with status: 24
2026-04-24T13:18:42.203+05:30 ORA-27301: OS failure message: Too many open
files
ORA-12547: TNS:lost contact
```

Issue: Semaphore/shared-memory exhaustion in a VM guest
Example snippet ID: `SYS-02`
Environment: VMware/KVM
Type: Synthetic
Why representative: Uses real Unix IPC tooling style and Oracle's generic OS-call failure family; the exact failing syscall text is the part the framework should key on. [35]

```
$ ipcs -ls
------ Semaphore Limits --------
```

```
max number of arrays = 128
max semaphores per array = 250
semget: No space left on device
2026-04-24T13:41:05.733+05:30 Errors in file /u01/app/oracle/diag/rdbms/<DB>/
<INST>/trace/<INST>_ora_25110.trc:
2026-04-24T13:41:05.733+05:30 ORA-27300: OS system dependent operation:semget
failed with status: 28
2026-04-24T13:41:05.734+05:30 ORA-27301: OS failure message: No space left on
device
```

Issue: Container with low `memlock` / no HugePages headroom at startup
Example snippet ID: `SYS-03`
Environment: Docker/K8s
Type: Synthetic
Why representative: This snippet intentionally uses command-output style plus Oracle startup failure because that is how this class of issue is usually debugged in containers; the actual container or pod manifest is the companion artefact, not the main evidence. [36]

```
$ ulimit -l
64
$ grep HugePages /proc/meminfo
HugePages_Total:       0
HugePages_Free:        0
2026-04-24T14:05:11.019+05:30 Errors in file /u01/app/oracle/diag/rdbms/<DB>/
<INST>/trace/<INST>_ora_11001.trc:
2026-04-24T14:05:11.019+05:30 ORA-27102: out of memory
2026-04-24T14:05:11.020+05:30 ORA-04030: out of process memory when trying to
allocate 65536 bytes (ksmcre,ksmchdl)
```

Issue: Hung task in kernel path, Oracle process blocked in uninterruptible I/O
Example snippet ID: `SYS-04`
Environment: physical bare-metal
Type: Synthetic
Why representative: The `INFO: task <process>:<pid> blocked for more than <N> seconds` pattern is documented in Red Hat guidance and is a strong host-side signal for system/path stalls that later surface as Oracle hangs or disconnects. [37]

```
Apr 24 14:29:44 host-a kernel: INFO: task oracle:24173 blocked for more than
120 seconds.
Apr 24 14:29:44 host-a kernel: "echo 0 > /proc/sys/kernel/
hung_task_timeout_secs" disables this message.
Apr 24 14:29:44 host-a kernel: task:oracle         state:D stack:0     pid:
24173 ppid:1 flags:0x00004000
Apr 24 14:29:44 host-a kernel: Call Trace:
Apr 24 14:29:44 host-a kernel:  __schedule+0x2d1/0x870
Apr 24 14:29:44 host-a kernel:  io_schedule+0x42/0x70
```

```
ORA-03113: end-of-file on communication channel
ORA-03135: connection lost contact
```

Network snippets

Network incidents usually describe a **broken conversation**, not the underlying reason. The host-layer evidence is packet drop, conntrack saturation, vSwitch receive/transmit drops, or bandwidth-ceiling signals from the cloud provider; Oracle then emits `TNS-12537`, `ORA-03135`, `ORA-12547`, or `ORA-12170`. [38]

Issue: Conntrack table full causing packet drops and TNS disconnects
Example snippet ID: `NET-01`
Environment: physical bare-metal
Type: Synthetic
Why representative: The kernel message is canonical, and Oracle listener / client close semantics map cleanly to it. [39]

```
Apr 24 15:02:18 db-gw kernel: nf_conntrack: table full, dropping packet
Apr 24 15:02:19 db-gw kernel: nf_conntrack: table full, dropping packet
Apr 24 15:02:19 db-gw kernel: nf_conntrack: table full, dropping packet
24-APR-2026 15:02:19 * (CONNECT_DATA=(SERVICE_NAME=<SERVICE_NET1>)) *
(ADDRESS=(PROTOCOL=tcp)(HOST=<IP_01>)(PORT=55122)) * establish *
<SERVICE_NET1> * 12537
TNS-12537: TNS:connection closed
TNS-00507: Connection closed
Linux Error: 104: Connection reset by peer
ORA-03135: connection lost contact
```

Issue: vSwitch / guest receive-drop problem in VMware
Example snippet ID: `NET-02`
Environment: VMware
Type: Synthetic
Why representative: Uses official `%DRPTX` terminology from esxtop plus realistic guest ethtool error counters; Oracle sees an abrupt partner loss. [40]

```
ESXTOP-NET 2026-04-24 15:18:00
PORT-ID  USED-BY        PKTTX/s  PKTRX/s  %DRPTX
33554479 oracle-db-vm   44190    43812     2.7
ethtool -S ens160
rx_packets: 1707183512
rx_dropped: 1325896
24-APR-2026 15:18:03 * (CONNECT_DATA=(SERVICE_NAME=<SERVICE_NET2>)) *
establish * <SERVICE_NET2> * 12547
TNS-12547: TNS:lost contact
```

Issue: Kubernetes node networking pressure / pod-facing timeout
Example snippet ID: `NET-03`
Environment: Docker/K8s

Type: Synthetic
Why representative: Uses Kubernetes node logging model and Oracle timeout text. In practice, node conntrack or CNI pressure often shows up to the DBA only as intermittent connect timeout. [41]

```
Apr 24 15:44:27 k8s-node-03 kernel: nf_conntrack: table full, dropping packet
Apr 24 15:44:27 k8s-node-03 kubelet[909]: Readiness probe failed: dial tcp
10.42.3.18:1521: i/o timeout
$ kubectl logs oracle-db-0 --previous --tail=3
ORA-12170: Cannot connect. TCP connect timeout of 3 seconds exceeded
ORA-03135: connection lost contact
$ kubectl events --for pod/oracle-db-0
15s Warning Unhealthy pod/oracle-db-0 Readiness probe failed: timeout
12s Warning BackOff   pod/oracle-db-0 Back-off restarting failed container
```

Issue: GCP instance-level bandwidth ceiling reached
Example snippet ID: `NET-04`
Environment: GCP
Type: Synthetic
Why representative: Google explicitly documents that bandwidth is enforced per instance, not per vNIC. The reviewed sources did not provide one exact Cloud Monitoring metric name for this specific formatted line, so the snippet keeps the dashboard export label generic and marks the metric identifier implicitly as environment-specific. [42]

```
2026-04-24T10:25:00Z cloud=gcp dashboard=instance-network-bandwidth
egress_gbps=22.8 limit_gbps=23.0
2026-04-24T10:25:00Z cloud=gcp note=exact Cloud Monitoring metric name
unspecified in reviewed sources
2026-04-24T15:55:01+05:30 sar[1881]: IFACE   rxpck/s txpck/s  rxkB/s  txkB/s
rxdrop/s txdrop/s
2026-04-24T15:55:01+05:30 sar[1881]: ens4    43812.0 44190.0 17283.4
22591.0     0.0    317.0
24-APR-2026 15:55:04 * (CONNECT_DATA=(SERVICE_NAME=<SERVICE_NET4>)) *
establish * <SERVICE_NET4> * 12170
ORA-12170: Cannot connect. TCP connect timeout of 3 seconds exceeded
ORA-03135: connection lost contact
TNS-12537: TNS:connection closed
```

Mapping and comparison tables

The first table is the requested mapping index. It is an index over the cited snippet corpus above, so each row inherits the evidence and citations attached to its snippet explanation.

Issue	Log type	Example snippet ID	Oracle alert/ trace	Environment	Notes
CPU saturation, connect timeout	PSI + vmstat + listener log	CPU-01	`ORA-3136`, `12535`	physical	Telemetry-first CPU issue; no hard kernel error
CPU ready / co-stop	esxtop CPU + listener log	CPU-02	`ORA-3136`, `TNS-12547`	VMware	VM scheduling delay
Steal time	top/vmstat + listener/client	CPU-03	`ORA-3136`, `ORA-03135`, `TNS-12547`	KVM	Host overcommit exposed as `%st`
Cgroup CPU throttling	`cpu.stat` + listener log	CPU-04	`ORA-3136`, `12535`	Docker/K8s	Quota-induced delay
Host OOM kill	dmesg/journal + ADR trace	MEM-01	`ORA-04030`, `ORA-03113`	physical	Strong causal correlation
Balloon/ swap pressure	esxtop memory + ADR trace	MEM-02	`ORA-27102`, `ORA-04030`	VMware	Guest memory reclaimed
Container OOM	pod state + `memory.events` + Oracle log	MEM-03	`ORA-04030`, `ORA-03113`	Docker/K8s	High-value training sample
Cloud VM guest OOM	guest journal + ADR trace	MEM-04	`ORA-27102`, `ORA-04030`, `ORA-03113`	Azure	Provider host path not required
Hard NVMe / block error	dmesg + ADR trace	IO-01	`ORA-27072`, `ORA-01114`	physical	Media/path failure
Storage latency	esxtop storage + SQL trace waits	IO-02	wait text, no ORA	VMware	Latency-only incident
Ephemeral storage full	kube event + filesystem + ADR trace	IO-03	`ORA-19502`, `ORA-00257`	Docker/K8s	Space exhaustion, not media fault
Cloud block latency	CloudWatch metrics + SQL trace waits	IO-04	wait text, no ORA	AWS	Queue/latency issue
Open-file exhaustion	shell limit + ADR trace	SYS-01	`ORA-27300`, `ORA-27301`, `ORA-12547`	physical	Straightforward errno mapping

Issue	Log type	Example snippet ID	Oracle alert/ trace	Environment	Notes
IPC semaphore exhaustion	`ipcs` + ADR trace	SYS-02	`ORA-27300`, `ORA-27301`	VMware/ KVM	Sysctl / IPC sizing issue
Memlock / HugePages startup issue	shell output + ADR trace	SYS-03	`ORA-27102`, `ORA-04030`	Docker/K8s	Container launch configuration issue
Hung task / D-state	kernel log + client symptom	SYS-04	`ORA-03113`, `ORA-03135`	physical	Kernel-side stall evidence
Conntrack full	kernel log + listener/client	NET-01	`TNS-12537`, `ORA-03135`	physical	Packet drops cause disconnect
vSwitch / guest drops	esxtop net + ethtool + listener	NET-02	`TNS-12547`	VMware	Hypervisor / guest NIC contention
Node network pressure	node log + pod logs + events	NET-03	`ORA-12170`, `ORA-03135`	Docker/K8s	CNI/node issue surfaced to DB
Instance bandwidth ceiling	cloud dashboard + guest net + Oracle Net	NET-04	`ORA-12170`, `ORA-03135`, `TNS-12537`	GCP	Exact metric name unspecified

The comparison table below is intended for quick framework coverage review.

Snippet ID	Issue	Log type	Environment	Synthetic/ Official	Key indicators
CPU-01	CPU saturation	PSI, vmstat, listener	physical	Synthetic	`cpu.pressure`, high run queue, `ORA-3136`
CPU-02	CPU ready / co-stop	esxtop CPU, listener	VMware	Synthetic	`%RDY`, `%CSTP`, `TNS-12547`
CPU-03	Steal time	top/vmstat, client log	KVM	Synthetic	`%st`, timeout, `ORA-03135`
CPU-04	Cgroup throttling	`cpu.stat`, listener	Docker/K8s	Synthetic	`nr_throttled`, `throttled_usec`
MEM-01	Host OOM kill	dmesg, ADR trace	physical	Synthetic	`Out of memory`, `Killed process`, `ORA-04030`
MEM-02	Balloon/ swap	esxtop MEM, vmstat, ADR	VMware	Synthetic	`MCTL`, `SWCUR`, swap growth

Snippet ID	Issue	Log type	Environment	Synthetic/ Official	Key indicators
MEM-03	Pod OOMKill	pod state, `memory.events`, Oracle log	Docker/K8s	Synthetic	`Reason: OOMKilled`, `oom_kill 1`
MEM-04	Guest OOM in cloud VM	journal, ADR	Azure	Synthetic	guest OOM + Oracle EOF
IO-01	NVMe / block failure	dmesg, ADR trace	physical	Synthetic	`reset controller`, `blk_update_request`, `ORA-27072`
IO-02	Storage latency	esxtop storage, SQL trace	VMware	Synthetic	`DAVG/KAVG/GAVG`, long `WAIT` lines
IO-03	Ephemeral disk full	kube event, `df`, ADR trace	Docker/K8s	Synthetic	`Evicted`, `Use%=100`, `ORA-00257`
IO-04	EBS latency	CloudWatch metrics, SQL trace	AWS	Synthetic	`VolumeQueueLength`, `VolumeAvgWriteLatency`
SYS-01	`nofile` exhaustion	shell output, ADR trace	physical	Synthetic	`ulimit -n`, `Too many open files`
SYS-02	IPC limits	`ipcs`, ADR trace	VMware/ KVM	Synthetic	`semget`, `No space left on device`
SYS-03	`memlock` / HugePages gap	shell output, ADR trace	Docker/K8s	Synthetic	`ulimit -l`, `HugePages_Total: 0`
SYS-04	Hung task	kernel log, client symptom	physical	Synthetic	`blocked for more than 120 seconds`
NET-01	Conntrack saturation	kernel, listener/ client	physical	Synthetic	`nf_conntrack: table full`
NET-02	vSwitch / NIC drops	esxtop net, ethtool, listener	VMware	Synthetic	`%DRPTX`, `rx_dropped`
NET-03	K8s node networking issue	node log, pod log, events	Docker/K8s	Synthetic	readiness timeout, `ORA-12170`
NET-04	Per-instance bandwidth cap	cloud dashboard, `sar`, Oracle Net	GCP	Synthetic	egress near limit, drops, timeout

Artifact packaging and sanitization

A framework needs a **stable naming contract**, a simple directory layout, and metadata that explicitly records whether a snippet is synthetic or sanitised. Oracle ADR and `V$DIAG_INFO` give you canonical Oracle-side provenance; NIST log-management guidance and OWASP logging guidance support the need for disciplined retention, minimisation, and field-level masking. Oracle TFA's consistent masking of hostnames and IP addresses is a strong design model for the dataset. [43]

Recommended file naming and layout

Use terse IDs that are stable across schema versions.

```
diag-corpus/
  cpu/
    physical/
      CPU-01__cpu_saturation__physical__psi-vmstat-listener__v1.log
      CPU-01__cpu_saturation__physical__psi-vmstat-listener__v1.meta.yaml
    vmware/
      CPU-02__cpu_ready__vmware__esxtop-listener__v1.log
      CPU-02__cpu_ready__vmware__esxtop-listener__v1.meta.json
  memory/
    docker-k8s/
      MEM-03__cgroup_oom_kill__docker-k8s__pod-memoryevents-alert__v1.log
      MEM-03__cgroup_oom_kill__docker-k8s__pod-memoryevents-
alert__v1.meta.yaml
  disk-io/
    aws/
      IO-04__ebs_latency__aws__cloudwatch-sqltrace__v1.log
      IO-04__ebs_latency__aws__cloudwatch-sqltrace__v1.meta.json
```

Recommended naming pattern:

```
<ID>__<issue>__<environment>__<log-types>__v<version>.<ext>
```

Required metadata fields:

- `id`
- `category`
- `issue`
- `environment`
- `timestamp_pattern`
- `source_type`
- `sanitized`
- `original_source`
- `oracle_artifact`
- `os_or_platform_artifacts`
- `causal_chain`
- `key_indicators`

- `generation_method`
- `notes`

Use `original_source: unspecified` when the line is wholly synthetic or when the exact public origin cannot be published.

Metadata schema examples

Sample metadata for `CPU-02` in YAML

```yaml
id: CPU-02
category: cpu
issue: vmware_cpu_ready_costop
environment: vmware
timestamp_pattern: "mixed: esxtop wall-clock + Oracle listener date format"
source_type: synthetic
sanitized: true
original_source: "Oracle Net error text + Broadcom esxtop counter names"
oracle_artifact:
  type: listener_log
  path: "diag/tnslsnr/<host>/<listener>/alert/log.xml or text view (resolved
at runtime)"
os_or_platform_artifacts:
  - "esxtop batch export (user-defined filename)"
causal_chain:
  - "ESXi overcommit"
  - "high %RDY / %CSTP"
  - "Oracle process scheduling delay"
  - "listener handoff timeout / lost contact"
key_indicators:
  - "%RDY > normal"
  - "%CSTP > normal"
  - "ORA-3136"
  - "TNS-12547"
generation_method: "Synthetic lines composed from official counter names and
Oracle error texts"
notes: "Minimal unit-test sample; no real hostnames or PIDs"
```

Sample metadata for `MEM-03` in JSON

```json
{
  "id": "MEM-03",
  "category": "memory",
  "issue": "cgroup_oom_kill",
  "environment": "docker-k8s",
  "timestamp_pattern": "pod event relative time + Oracle ADR timestamp",
  "source_type": "synthetic",
  "sanitized": true,
  "original_source": "Linux cgroup v2 memory.events + Kubernetes pod
```

```
lifecycle + Oracle ORA-04030",
  "oracle_artifact": {
    "type": "alert_or_trace",
    "path": "diag/rdbms/<db>/<inst>/trace/<inst>_ora_<pid>.trc"
  },
  "os_or_platform_artifacts": [
    "/sys/fs/cgroup/memory.events",
    "kubectl describe pod",
    "kubectl logs --previous"
  ],
  "causal_chain": [
    "memory.max breach",
    "oom event",
    "oom_kill increments",
    "oracle process terminated",
    "ORA-03113"
  ],
  "key_indicators": [
    "Reason OOMKilled",
    "Exit Code 137",
    "oom_kill 1",
    "ORA-04030"
  ],
  "generation_method": "Synthetic composition preserving real field names",
  "notes": "Preferred training sample for container OOM classification"
}
```

Sample metadata for `IO-04` in YAML

```
id: IO-04
category: disk-io
issue: aws_ebs_queue_latency
environment: aws
timestamp_pattern: "CloudWatch ISO8601 + Oracle SQL trace TIM"
source_type: synthetic
sanitized: true
original_source: "AWS EBS metric names + Oracle SQL trace wait naming"
oracle_artifact:
  type: sql_trace
  path: "diag/rdbms/<db>/<inst>/trace/<inst>_ora_<pid>.trc"
os_or_platform_artifacts:
  - "CloudWatch metric export CSV (user-defined filename)"
causal_chain:
  - "EBS queue depth rise"
  - "read/write latency rise"
  - "db file/log write wait inflation"
  - "slow commit / slow read"
key_indicators:
  - "VolumeQueueLength"
  - "VolumeAvgReadLatency"
```

```
  - "VolumeAvgWriteLatency"
  - "db file sequential read"
generation_method: "Synthetic CSV lines paired with SQL trace waits"
notes: "Good cloud-storage latency benchmark sample"
```

Same `CPU-02` example in JSON

```
{
  "id": "CPU-02",
  "category": "cpu",
  "issue": "vmware_cpu_ready_costop",
  "environment": "vmware",
  "timestamp_pattern": "mixed: esxtop wall-clock + Oracle listener date
format",
  "source_type": "synthetic",
  "sanitized": true,
  "original_source": "Oracle Net error text + Broadcom esxtop counter names",
  "oracle_artifact": {
    "type": "listener_log",
    "path": "diag/tnslsnr/<host>/<listener>/alert/log.xml or text view
(runtime-resolved)"
  },
  "os_or_platform_artifacts": [
    "esxtop batch export (user-defined filename)"
  ],
  "causal_chain": [
    "ESXi overcommit",
    "high %RDY / %CSTP",
    "Oracle process scheduling delay",
    "listener handoff timeout"
  ],
  "key_indicators": ["%RDY", "%CSTP", "ORA-3136", "TNS-12547"],
  "generation_method": "Synthetic",
  "notes": "No production identifiers"
}
```

Same `MEM-03` example in YAML

```
id: MEM-03
category: memory
issue: cgroup_oom_kill
environment: docker-k8s
timestamp_pattern: "pod relative + ADR absolute"
source_type: synthetic
sanitized: true
original_source: "memory.events + Kubernetes pod lifecycle + Oracle error
text"
oracle_artifact:
  type: alert_or_trace
```

```yaml
  path: "diag/rdbms/<db>/<inst>/trace/<inst>_ora_<pid>.trc"
os_or_platform_artifacts:
  - "/sys/fs/cgroup/memory.events"
  - "kubectl describe pod"
causal_chain:
  - "memory.max breach"
  - "oom"
  - "oom_kill"
  - "oracle exit"
  - "ORA-03113"
key_indicators: ["OOMKilled", "Exit Code 137", "oom_kill 1", "ORA-04030"]
generation_method: "Synthetic"
notes: "Container memory failure signature"
```

Same `IO-04` example in JSON

```json
{
  "id": "IO-04",
  "category": "disk-io",
  "issue": "aws_ebs_queue_latency",
  "environment": "aws",
  "timestamp_pattern": "CloudWatch ISO8601 + Oracle SQL trace TIM",
  "source_type": "synthetic",
  "sanitized": true,
  "original_source": "AWS EBS metrics + Oracle SQL trace wait names",
  "oracle_artifact": {
    "type": "sql_trace",
    "path": "diag/rdbms/<db>/<inst>/trace/<inst>_ora_<pid>.trc"
  },
  "os_or_platform_artifacts": [
    "CloudWatch metric export CSV"
  ],
  "causal_chain": [
    "queue depth rise",
    "latency rise",
    "wait inflation",
    "slow commits"
  ],
  "key_indicators": [
    "VolumeQueueLength",
    "VolumeAvgReadLatency",
    "VolumeAvgWriteLatency",
    "log file sync"
  ],
  "generation_method": "Synthetic",
  "notes": "Cloud storage latency sample"
}
```

Sanitization rules, regexes, and checklist

Preserve **diagnostic fidelity**, not literal identity. The critical fields to preserve are: time sequencing, error text, counters, resource names (`nvme1c1n1`, `%RDY`, `VolumeQueueLength`, `oom_kill`), and Oracle file-role semantics (`trace`, `alert`, `listener`, `incident`). Oracle TFA's consistent masking model is the right design target. [44]

Recommended regex masks:

```
IPv4:
\b(?:\d{1,3}\.){3}\d{1,3}\b

IPv6:
\b(?:[A-Fa-f0-9]{1,4}:){2,7}[A-Fa-f0-9]{1,4}\b

Hostnames in key/value form:
(?<=\bHOST=)[A-Za-z0-9._-]+
(?<=\bhost=)[A-Za-z0-9._-]+

Generic FQDN hostnames:
\b(?:[A-Za-z0-9](?:[A-Za-z0-9-]{0,61}[A-Za-z0-9])?\.)+[A-Za-z]{2,63}\b

Usernames / OS users:
(?<=\bUSER=)[A-Za-z0-9._$#-]+
(?<=\bosuser=)[A-Za-z0-9._$#-]+
(?<=\buid=)[A-Za-z0-9._$#-]+

PIDs / SPIDs:
(?<=\bpid:)\d+
(?<=\bPID: )\d+
(?<=\bspid=)\d+
(?<=_ora_)\d+

Absolute Unix paths:
/(?:[A-Za-z0-9._-]+/)+[A-Za-z0-9._-]+

Oracle SID / service name:
(?<=\bSID=)[A-Za-z0-9_$#.-]+
(?<=\bSERVICE_NAME=)[A-Za-z0-9_$#.-]+

TNS address blocks:
\((?:HOST|PORT|SERVICE_NAME|SID)=[^)]+\)

Timestamps:
\b\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}(?:\.\d+)?(?:Z|[+-]\d{2}:\d{2})\b
\b[A-Z][a-z]{2}\s+\d{1,2}\s+\d{2}:\d{2}:\d{2}\b
```

Sanitization checklist:

- Replace IPs, hostnames, usernames, service names, SIDs, ports when they are identifying rather than diagnostic.
- Keep **relative time order**. If absolute timestamps are sensitive, tokenise them as `TS_001`, `TS_002`, ... in order.
- Preserve file-role semantics: keep `/diag/rdbms/<DB>/<INST>/trace/...` structure even if names are tokenised.
- Preserve device classes and forms: `nvme1c1n1`, `dm-0`, `ens160`, `naa.<token>`.
- Keep errno text and ORA/TNS text unchanged unless it itself contains identifiers.
- Remove SQL literals, bind values, client addresses, wallet paths, key material, and full connection descriptors unless strictly required.
- For cloud metrics, keep namespace + metric name where officially documented; mask instance IDs, volume IDs, project IDs, subscription IDs.
- Mark every processed artifact with `sanitized: true` and the sanitiser version.

Minimal Python sanitiser with deterministic token replacement:

```
#!/usr/bin/env python3
import re
import sys
from collections import defaultdict

text = sys.stdin.read()

patterns = {
    "ts": re.compile(r"\b\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}(?:\.\d+)?(?:Z|
[+-]\d{2}:\d{2})\b|\b[A-Z][a-z]{2}\s+\d{1,2}\s+\d{2}:\d{2}:\d{2}\b"),
    "ip": re.compile(r"\b(?:\d{1,3}\.){3}\d{1,3}\b"),
    "ipv6": re.compile(r"\b(?:[A-Fa-f0-9]{1,4}:){2,7}[A-Fa-f0-9]{1,4}\b"),
    "host_kv": re.compile(r"(?<=\bHOST=)[A-Za-z0-9._-]+|(?<=\bhost=)[A-Za-
z0-9._-]+"),
    "fqdn": re.compile(r"\b(?:[A-Za-z0-9](?:[A-Za-z0-9-]{0,61}[A-Za-z0-9])?
\.)+[A-Za-z]{2,63}\b"),
    "user": re.compile(r"(?<=\bUSER=)[A-Za-z0-9._$#-]+|(?<=\bosuser=)[A-Za-
z0-9._$#-]+"),
    "sid": re.compile(r"(?<=\bSID=)[A-Za-z0-9_$#.-]+|(?<=\bSERVICE_NAME=)[A-
Za-z0-9_$#.-]+"),
    "path": re.compile(r"/(?:[A-Za-z0-9._-]+/)+[A-Za-z0-9._-]+"),
    "pid": re.compile(r"(?<=\bpid:)\d+|(?<=\bPID: )\d+|(?<=\bspid=)\d+|(?
<=_ora_)\d+")
}

counters = defaultdict(int)
maps = defaultdict(dict)

def token(kind, value):
    if value not in maps[kind]:
        counters[kind] += 1
        maps[kind][value] = f"<{kind.upper()}_{counters[kind]:03d}>"
```

```
    return maps[kind][value]

order = ["ts", "ip", "ipv6", "host_kv", "fqdn", "user", "sid", "path", "pid"]

for kind in order:
    text = patterns[kind].sub(lambda m: token(kind, m.group(0)), text)

sys.stdout.write(text)
```

Generation and validation

The safest way to generate framework inputs is to separate **safe reproduction** from **unsafe fault injection**. Safe reproduction is preferred for CPU pressure, cgroup throttling, pod OOM kill, listener/session tracing, journald/syslog injection, and filesystem-full conditions in disposable test storage. Unsafe or environment-specific cases include real media errors, real SAN path failure, or deliberately polluting production kernel logs. Oracle itself warns that broad SQL tracing can impact performance, so use session-scoped tracing. [45]

Safe generation and simulation commands

Goal	Safe generation method	Unsafe / unspecified	Notes
Inject journal/syslog-style line	`logger -t diag-fw 'nf_conntrack: table full, dropping packet'`	Writing directly to `/dev/kmsg` on shared hosts	Use `logger` unless you are in a disposable VM
Disposable kmsg write for dmesg-like test data	`echo '<3>nf_conntrack: table full, dropping packet' | sudo tee /dev/kmsg`	Unsafe on production; pollutes kernel log	Prefer only in isolated VM
CPU pressure	`stress-ng --cpu 4 --timeout 60s` plus capture `/proc/pressure/cpu`, `vmstat 1`	Running against production DB host	Creates real CPU telemetry without fabricating counters
Container CPU throttling	`docker run --cpus=0.5 ...` and read `cpu.stat`	Tight quotas on production DB containers	Good for `CPU-04`-like samples
Safe OOM in disposable scope	`systemd-run --scope -p MemoryMax=256M bash -c 'stress-ng --vm 1 --vm-bytes 512M --timeout 20s'`	Host-wide OOM on shared machines	Prefer cgroup-scoped failure, not host-wide exhaustion
K8s OOMKill	run a limited test pod with `resources.limits.memory`, then allocate beyond limit	Any production namespace	Yields real `OOMKilled` + `memory.events`

Goal	Safe generation method	Unsafe / unspecified	Notes
Filesystem-full / archive-full	loopback file or small tmpfs mounted under a test path, then fill it	Filling a live FRA or data volume	Safely generates `No space left on device`
Docker/K8s disk pressure	use tiny PV/emptyDir/loopback-based mount in test cluster	Using writable image layer or shared node rootfs on production	Good for `ORA-00257` / `ORA-19502` style simulations
Real block-I/O error	**Prefer synthetic replay** of kernel-style lines or disposable VM with environment-specific fault injector	Intentionally causing real disk path/ media fault on production	Exact safe kernel-fault injector choice is environment-specific and otherwise unspecified
Conntrack saturation	isolated VM or namespace; optionally lower conntrack max in test environment and create many short-lived connections	Touching shared-node conntrack sysctls	Strongly isolate if you do this for real
Oracle trace generation	session-scoped SQL trace + `V$DIAG_INFO` lookup	instance-wide trace in production	Safer and officially supported

Minimal Oracle-side generation steps:

```
-- session-scoped trace; do not enable instance-wide trace casually
ALTER SESSION SET tracefile_identifier = 'FW_TEST_01';
ALTER SESSION SET sql_trace = TRUE;

-- find active trace file
SELECT value
FROM   v$diag_info
WHERE  name = 'Default Trace File';

-- optional network / IPC dump in a lab environment
-- oradebug setmypid
-- oradebug ipc

-- generate a checkpoint-related alert/log boundary in a lab
ALTER SYSTEM CHECKPOINT;

-- controlled session kill in a lab only
ALTER SYSTEM KILL SESSION 'sid,serial#' IMMEDIATE;
```

These commands are aligned with Oracle's documented ADR, SQL trace, and `ALTER SYSTEM KILL SESSION` facilities. `oradebug ipc` is documented in Oracle RAC troubleshooting as writing a trace file under `DIAGNOSTIC_DEST`. [46]

Minimal shell helpers:

```
# CPU pressure sample
stress-ng --cpu 4 --timeout 30s &
vmstat 1 5
cat /proc/pressure/cpu

# Safe journal injection
logger -t diag-fw 'WARNING: inbound connection timed out (ORA-3136)'

# Disposable cgroup-scoped OOM (systemd user scope)
systemd-run --user --scope -p MemoryMax=256M bash -c 'stress-ng --vm 1 --vm-
bytes 512M --timeout 15s'

# Small loopback filesystem to simulate "disk full"
truncate -s 512M /tmp/fill.img
mkfs.ext4 -F /tmp/fill.img
sudo mount -o loop /tmp/fill.img /mnt/testfill
dd if=/dev/zero of=/mnt/testfill/blob bs=1M status=progress || true
df -h /mnt/testfill

# Docker event capture
docker events --since 10m

# Kubernetes events
kubectl events --types=Warning
kubectl logs -p oracle-db-0
```

Authenticity validation commands

Use the checks below to decide whether a real snippet actually matches the claimed issue. Oracle and platform evidence should agree on **time window**, **resource class**, and **error family**.

Category	What to validate in a real log	Quick validation commands
CPU	High scheduler pressure or wait-to-run; Oracle Net timeout or slow session creation in same window	`cat /proc/pressure/cpu`; `vmstat 1`; `pidstat -u 1`; VMware `esxtop` checking `%RDY/%CSTP/%MLMTD`; KVM `top` / `vmstat` `%st`

Category	What to validate in a real log	Quick validation commands
Memory	OOM line or `memory.events` increment plus Oracle `ORA-04030` / `ORA-27102` or disconnected client	`journalctl -k | grep -Ei 'out of memory|killed process'`; `cat /sys/fs/cgroup/memory.events`; `grep HugePages /proc/meminfo`; `adrci show alert -tail`; `SELECT value FROM v$diag_info WHERE name='Default Trace File';`
Disk/I/O	Same device / mount / volume implicated across kernel, VM/cloud, and Oracle wait/error	`dmesg -T | grep -Ei 'blk_update_request|nvme|I/O error'`; `iostat -xz 1`; `cat /proc/pressure/io`; VMware `esxtop` for `DAVG/KAVG/GAVG`; AWS check `VolumeQueueLength`, `VolumeAvgReadLatency`, `VolumeAvgWriteLatency`; GCP use disk queue-depth / latency dashboard labels; Azure use disk queue-depth / latency metrics
Kernel/ system limits	Oracle `ORA-27301` OS message matches the actual shell/sysctl/ IPC limit observed	`ulimit -a`; `sysctl -a | egrep 'kernel.sem|fs.file-max|ip_local_port_range'`; `ipcs -ls`; `grep HugePages /proc/meminfo`; `adrci show alert -tail`
Network	Packet drop / conntrack / NIC or cloud allowance evidence aligns with listener/client timeout/disconnect	`dmesg -T | grep -i conntrack`; `ss -s`; `sar -n DEV,TCP,ETCP 1`; `ethtool -S <if>`; VMware `esxtop` network view `%DRPTX`; AWS ENA allowances such as `bw_in_allowance_exceeded` / CloudWatch-agent `ethtool_bw_in_allowance_exceeded`; for GCP/Azure exact metric names are partly environment-specific and **unspecified in the reviewed sources**, so validate via provider dashboard export plus guest counters

The provider-specific validation names that are strongest in the reviewed sources are AWS EBS `VolumeQueueLength`, `VolumeAvgReadLatency`, `VolumeAvgWriteLatency`, and ENA allowance metrics such as `bw_in_allowance_exceeded` / `ethtool_bw_in_allowance_exceeded`. GCP explicitly documents per-instance bandwidth ceilings and disk queue-depth / latency dashboard labels, and Azure explicitly documents disk IO, throughput, queue depth, and latency metrics; however, the exact single API metric name needed for every synthetic line above is partly **unspecified in the reviewed sources**, so capture dashboard-export text or API descriptors alongside the snippet. [47]

Legal and ethical note

Do not place raw production Oracle or OS logs into a training or test corpus unless you have explicit authority, a retention basis, and a masking process. NIST log-management guidance emphasises disciplined log retention and handling, OWASP emphasises excluding sensitive material from logs, and Oracle TFA explicitly masks hostnames and IP addresses while preserving diagnostic usefulness. For a framework corpus, the safe default is **synthetic first, sanitised second, raw never by default**. [48]

Recommended practice:

- keep only the minimum lines needed for unit tests;

- record provenance and sanitiser version in metadata;
- strip or tokenise PII, secrets, connection descriptors, SQL literals, and customer identifiers;
- use time-bounded retention for any production-derived intermediate files;
- keep consent / approval records wherever internal policy requires them;
- publish only sanitised corpora, never raw support bundles.

Prioritised source stack for snippet sourcing

Use sources in this order when expanding the corpus:

Priority	Source type	Examples
highest	Oracle official	ADR / `DIAGNOSTIC_DEST`, `V$DIAG_INFO`, Oracle Net ADR/logging/tracing, Oracle Error Help, SQL trace and `ALTER SYSTEM` docs [49]
highest	Linux kernel official	PSI, cgroup v2 `memory.events`, OOM handling, `nf_conntrack` sysctls [50]
high	VMware / Broadcom official KB and docs	`esxtop` CPU/network/storage counters, `vmkernel.log` paths, VM log paths [51]
high	AWS official	EBS CloudWatch metrics, ENA allowance metrics, EC2 network ceilings [52]
high	GCP official	Persistent Disk performance and queue-depth/latency guidance, per-instance network bandwidth limits [53]
high	Azure official	VM/disk queue depth and latency metrics, Accelerated Networking behaviour [54]
medium	Kubernetes and Docker official	pod/node logging paths, events, previous logs, container writable-layer caveats, `docker events` [55]
medium	Red Hat official docs / KB	KVM steal time, balloon driver, representative Linux kernel error wording in enterprise environments [56]
lower	Third-party blogs	Use only when a primary source does not give a format example; mark as lower priority and keep synthetic attribution explicit

Where a requirement cannot be met exactly — such as a fixed host-log path for cloud provider metrics, or a universally documented flat-file `listener.log` location when ADR is enabled — the correct handling is to mark it **unspecified** and capture the nearest authoritative alternative: ADR path resolution, dashboard/API export, or guest-level logs. [57]

---

[1] [5] [17] [49] https://docs.oracle.com/en/database/oracle/oracle-database/19/refrn/DIAGNOSTIC_DEST.html
https://docs.oracle.com/en/database/oracle/oracle-database/19/refrn/DIAGNOSTIC_DEST.html

[2] [57] https://docs.oracle.com/en/database/oracle/oracle-database/26/netag/oracle-net-services-log-file-names.html
https://docs.oracle.com/en/database/oracle/oracle-database/26/netag/oracle-net-services-log-file-names.html

3 44 https://docs.oracle.com/en/engineered-systems/health-diagnostics/trace-file-analyzer/tfaug/quick-start-guide.html
https://docs.oracle.com/en/engineered-systems/health-diagnostics/trace-file-analyzer/tfaug/quick-start-guide.html

4 9 22 36 https://www.kernel.org/doc/html/latest/admin-guide/cgroup-v2.html
https://www.kernel.org/doc/html/latest/admin-guide/cgroup-v2.html

6 https://docs.oracle.com/en/database/oracle/oracle-database/23/netag/understanding-automatic-diagnostic-repository.html
https://docs.oracle.com/en/database/oracle/oracle-database/23/netag/understanding-automatic-diagnostic-repository.html

7 https://www.freedesktop.org/software/systemd/man/latest/systemd-journald.service.html
https://www.freedesktop.org/software/systemd/man/latest/systemd-journald.service.html

8 18 19 50 https://www.kernel.org/doc/html/latest/accounting/psi.html
https://www.kernel.org/doc/html/latest/accounting/psi.html

10 https://access.redhat.com/solutions/5716951
https://access.redhat.com/solutions/5716951

11 33 34 35 https://docs.oracle.com/en/error-help/db/ora-27300/index.html
https://docs.oracle.com/en/error-help/db/ora-27300/index.html

12 38 39 https://access.redhat.com/solutions/8721
https://access.redhat.com/solutions/8721

13 https://knowledge.broadcom.com/external/article/306962/location-of-esxi-log-files.html
https://knowledge.broadcom.com/external/article/306962/location-of-esxi-log-files.html

14 21 56 https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/virtualization_deployment_and_administration_guide/sect-kvm_guest_timing_management-steal_time_accounting
https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/virtualization_deployment_and_administration_guide/sect-kvm_guest_timing_management-steal_time_accounting

15 55 https://kubernetes.io/docs/concepts/cluster-administration/logging/
https://kubernetes.io/docs/concepts/cluster-administration/logging/

16 32 47 52 https://docs.aws.amazon.com/ebs/latest/userguide/ebs-io-characteristics.html
https://docs.aws.amazon.com/ebs/latest/userguide/ebs-io-characteristics.html

20 https://knowledge.broadcom.com/external/article/387750/understanding-the-cpu-ready-values-in-th.html
https://knowledge.broadcom.com/external/article/387750/understanding-the-cpu-ready-values-in-th.html

23 https://docs.oracle.com/en/engineered-systems/health-diagnostics/autonomous-health-framework/ahfug/ora-04030.html
https://docs.oracle.com/en/engineered-systems/health-diagnostics/autonomous-health-framework/ahfug/ora-04030.html

24 https://docs.kernel.org/6.1/mm/oom.html
https://docs.kernel.org/6.1/mm/oom.html

25 https://knowledge.broadcom.com/external/article/326270/impact-of-virtual-machine-memory-and-cpu.html
https://knowledge.broadcom.com/external/article/326270/impact-of-virtual-machine-memory-and-cpu.html

26 https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/
https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/

27 54 https://learn.microsoft.com/en-us/azure/virtual-machines/disks-metrics
https://learn.microsoft.com/en-us/azure/virtual-machines/disks-metrics

28 https://docs.oracle.com/en/error-help/db/ora-27072/
https://docs.oracle.com/en/error-help/db/ora-27072/

29 https://access.redhat.com/solutions/6589401
https://access.redhat.com/solutions/6589401

30 https://knowledge.broadcom.com/external/article/344099/using-esxtop-to-identify-storage-perform.html
https://knowledge.broadcom.com/external/article/344099/using-esxtop-to-identify-storage-perform.html

31 https://kubernetes.io/docs/concepts/storage/ephemeral-storage/
https://kubernetes.io/docs/concepts/storage/ephemeral-storage/

37 https://access.redhat.com/solutions/60572
https://access.redhat.com/solutions/60572

40 https://knowledge.broadcom.com/external/article/312565/the-output-of-esxtop-shows-dropped-recei.html
https://knowledge.broadcom.com/external/article/312565/the-output-of-esxtop-shows-dropped-recei.html

41 https://kubernetes.io/docs/concepts/cluster-administration/system-logs/
https://kubernetes.io/docs/concepts/cluster-administration/system-logs/

42 https://docs.cloud.google.com/compute/docs/network-bandwidth
https://docs.cloud.google.com/compute/docs/network-bandwidth

43 46 https://docs.oracle.com/en/database/oracle/oracle-database/19/refrn/V-DIAG_INFO.html
https://docs.oracle.com/en/database/oracle/oracle-database/19/refrn/V-DIAG_INFO.html

45 https://docs.oracle.com/en/database/oracle/oracle-database/23/refrn/SQL_TRACE.html
https://docs.oracle.com/en/database/oracle/oracle-database/23/refrn/SQL_TRACE.html

48 https://csrc.nist.gov/pubs/sp/800/92/final
https://csrc.nist.gov/pubs/sp/800/92/final

51 https://knowledge.broadcom.com/external/article/308290/esxtop-overview-for-performance-troubles.html
https://knowledge.broadcom.com/external/article/308290/esxtop-overview-for-performance-troubles.html

53 https://docs.cloud.google.com/compute/docs/disks/performance
https://docs.cloud.google.com/compute/docs/disks/performance